

First, Do No Harm To the Data: RAG and Dual-LLM Architecture for Medical Agents

Anonymous

Abstract—

*Index Terms—*Clinical-Decision Making, Large Language Models, LLM-as-a-Judge

I. INTRODUCTION

Modern medical AI systems increasingly rely on Retrieval-Augmented Generation (RAG), where an AI model retrieves information from external sources such as research databases, health forums, and clinical websites before generating an answer. While this approach improves knowledge coverage, it also introduces serious safety and security risks. The core problem is that current RAG-based medical agents do not control how retrieved information influences reasoning and decision-making. As a result, low-quality, misleading or even malicious content can shape medical conclusions, recommendations, or tool actions.

In particular, these systems treat all retrieved text as neutral context, without distinguishing between high-trust sources like peer-reviewed research and low-trust sources like anonymous online forums. Furthermore, even when planning and reasoning components are isolated, there is typically no mechanism that controls how retrieved data flows through the system or how it affects sensitive actions such as treatment recommendation or external tool calls. Therefore, there is a clear need for a more secure architecture that can track the origin of retrieved information and enforce strict policies on how that information is allowed to influence medical reasoning and actions.

To understand the problem more clearly, imagine a medical AI assistant that receives a patient's question. The system retrieves information from different sources, such as academic research articles, clinical guidelines, and online patient discussion boards. All of this retrieved text is then given to the language model, which uses it to generate a diagnosis or treatment suggestion.

At first glance this seems powerful and helpful. However, the problem begins with the assumption that all retrieved information can be treated equally. In reality, a peer-reviewed clinical study and an anonymous forum comment do not carry the same level of reliability. Yet once they are placed into the model's context window, the system does not inherently understand these differences in trust. As a result, incorrect information may

influence clinical reasoning just as much as validated medical evidence.

The situation becomes even more concerning when consider prompt injection. Suppose someone posts malicious or misleading instructions inside a public health forum. If the RAG system retrieves that content, the language model may unintentionally incorporate those instructions into its reasoning process. Even if the overall task planning remains correct, the data being used to make decisions has already been compromised. In other words the control flow maybe safe, but the data flow is not.

This distinction between control flow and data flow is crucial. Protecting control flow means ensuring that the system follows the correct sequence of steps. Protecting data flow means ensuring that the information used within those steps is trustworthy and allowed to influence certain outcomes. In medical contexts, unsafe data flow can lead to incorrect diagnosis, inappropriate treatment suggestions, or unauthorized access to sensitive tools such as prescription system or medical records.

Another important aspect of the problem is action sensitivity. Medical AI agents may not only generate text but also interact with tools. They might write notes to electronic health records, generate prescriptions or send reports. If untrusted retrieved information influences these actions, the consequences are much more serious than simply generating a wrong sentence. The system could produce harmful clinical decisions or violate privacy constraints.

Current defense strategies, such as isolating planning models from data/processing models, improve security but do not fully solve this issue. They focus mainly on preventing malicious instructions from alerting the system's plan. However, they do not prevent malicious or low-quality data from shaping the content of decisions within that plan. In other words, they protect how the system acts, but not what information it uses to decide.

Therefore, the fundamental research challenge is to design a medical RAG architecture that does not merely retrieve and generate, but also understands where each piece of information comes from and enforce rules about how it can be used. The system must be able to differentiate between trusted and untrusted sources, track how retrieved data influences reasoning, and block

or restrict actions when that influence violates safety policies.

In summary, the problem is not simply about improving accuracy. It is about ensuring that medical AI systems remain safe, robust, and trustworthy when interacting with diverse and potentially unreliable external knowledge sources. Without mechanism to control data flow and influence, RAG based medical agents remain vulnerable to subtle but potentially dangerous failures.

a) *Contribution*: The main contributions of this paper are:

II. PRELIMINARIES

Retrieval-Augmented Generation (RAG) systems combine a retrieval module (which fetches relevant documents or knowledge) with a generative language model (LLM). This enables the LLM to ground its responses in external, possibly up-to-date, or domain-specific information. [1]

III. RELATED WORK

Digital healthcare is changing how patients receive care through technologies like wearable devices and AI tools. These allow doctors to monitor patients remotely, create personalized treatments, and predict health issues [2]. ML tracks how diseases progress, while language models understand medical notes to support better decisions. Together, these tools make healthcare more efficient, accurate, and accessible [3]. Dual LLM system splits an AI assistant into a privileged LLM (talks to trusted sources and uses tools) and a quarantined LLM (used anytime on untrusted content and expected to go rogue anytime) that can safely use tools and private data while mitigating prompt injection attacks [4].

IV. FIRST, DO NO HARM TO THE DATA

A. Agent Roles

V. METHODOLOGY

A. Query Formulation

The first stage of our methodology focuses on transforming raw user input into a clinically precise and retrieval-ready query. Since medical questions from users are often informal, emotional, or loosely structured, directly using them for retrieval can reduce accuracy and introduce ambiguity. Therefore, we design a multi-step query formulation pipeline that gradually refines the input while preserving its meaning.

- 1) **Intent Separation (LLM)**: We begin with intent separation. When a user describes their symptoms, they often include conversational elements such as filler words, personal concerns, or emotional expressions. For example, a user might say, ".....".

While this conveys important context, it also contains lexical noise that can negatively affect retrieval performance. In this step an LLM carefully rewrites the input into clinically focused language without changing the underlying meaning. The goal is not to reinterpret or diagnose, but simply to clarify and normalize the expression. As a result, the system produces a cleaner and medically oriented version of the query that retains semantic fidelity while reducing ambiguity.

- 2) **Symptom & Attribute Extraction (LLM)**: Once the query has been clarified, the next step is symptom and attribute extraction. At this stage, the LLM identifies explicitly mentioned clinical entities such as symptoms, duration, severity, age, gender, and any other relevant modifiers. Instead of working with free-form text, the system now begins converting the information into structured medical variables. For example, This transformation is important because structured variables are more stable and suitable for downstream reasoning and retrieval. It also reduces the risk that later stages of the system rely on vague or unstructured representations.
- 3) **Structured Representation (Deterministic)**: After extraction, the pipeline moves to structured representation, which is fully deterministic. Unlike previous steps, this stage does not involve further LLM reasoning. Instead, the extracted information is mapped into a canonical schema with predefined fields. The purpose of this step is threefold. First, it clearly separates clinical presentation from meta-information such as emotional tone or background context. Second, it provides a stable and standardized structure that can be consistently used for retrieval, evaluation, and auditing. Third, by removing generative flexibility at this stage, it prevents hallucinated details from propagating further into the system. In other words, once the information is structured, it becomes fixed and transparent.
- 4) **Query Generator (Deterministic)**: With structured data in place, the system proceeds to query generation, which is also deterministic. Here, the goal is to convert the structured clinical variables into retrieval-optimized search queries without introducing new semantic variation. This ensures that the meaning remains consistent with the original user intent. Two types of queries are generated in parallel. The first is a dense retrieval query, which takes the normalized symptoms and modifiers and reformulates them into a natural clinical sentence. This version is designed for embedding-based vector search. The second is a sparse retrieval query, which converts the same structured variables into a boolean-style keyword query suitable for

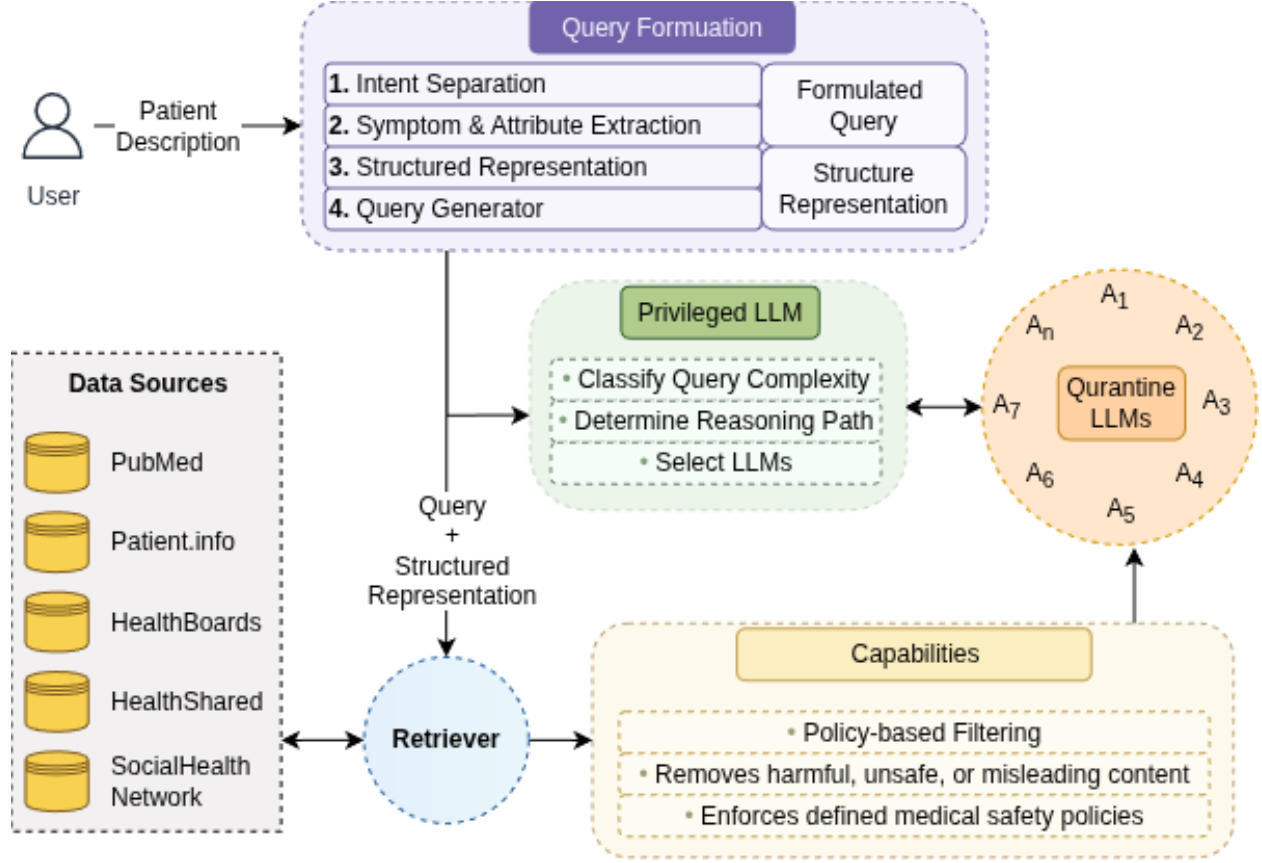


Fig. 1: Proposed CaMel Multi-Agent RAG Framework for Medical Query Processing. The system consists of a Query Formulation module, structured representation generation, RAG retrieval from multiple medical data sources, privileged LLM reasoning, moderation, quarantine LLMs, and policy-based safety enforcement.

traditional lexical search methods. By keeping this stage deterministic, we ensure that no unintended reinterpretation occurs.

- 5) **Dense + Sparse Retrieval Query:** Finally, the system performs hybrid retrieval using both dense and sparse queries. Medical queries often suffer from vocabulary mismatch. Patients may use everyday language, whereas scientific literature uses technical terminology. Dense retrieval helps bridge this gap by capturing semantic similarity through vector embeddings. At the same time, sparse retrieval method such as BM25 remain highly effective for exact keyword matching, especially for specific medical terms. Therefore, the dense reformulated query is embedded and used for vector search, while the sparse boolean query is used for key-word-based retrieval. The result from both methods are then merged or re-ranked to produce a final candidate set of documents.

By combining these steps in a structured and sequential manner, the query formulation stage en-

sures that retrieval begins with a clean, clinically grounded, and semantically stable representation of the user's concern. This careful preparation not only improves retrieval quality but also strengthens the overall robustness and safety of the system.

B. RAG over External Medical Sources

Once the query has been carefully formulated and structured, the system moves to the retrieval stage. At this point, the input is no longer informal user text. Instead, it is a clean, clinically organized representation that is optimized for searching medical knowledge sources. This makes the retrieval process more accurate and controlled.

TO begin with, the system sends the generated dense and sparse queries to external data sources. These sources may include: Pubmed, patientinfo, healthboards, healthsahred, and socialhealth networks. Because the query has already been cleaned and normalized, the search process is more focused and less likely to retrieve irrelevant content.

First, the dense retrieval query is used for semantic search. The system convert this query into a vector embedding and compares it with pre-indexed document embeddings stored in a vector database. This allows the system to find documents that are conceptually similar, even if they do not use the exact same wording. For example, if a patient describe "....." the system can still retrieve documents discussing "....." because the embedding captures meaning rather than just keywords. This step helps reduce the gap between everyday language and medical terminology.

At the same time, the sparse retrieval query is applied using traditional keyword-based search methods such as BM25. This approach focuses on exact term matching and is especially useful when the query includes specific medical terms, drug names, or well-defined conditions. While dense retrieval captures meaning, sparse retrieval ensures precision. By running both searches in parallel, the system benefits from the strength of each method.

After both retrieval processes are complete, the results are combined. The system may merge the document list or re-rank them based on relevance score. At this stage, the goal is to select a balanced and high-quality set of candidate documents. The system also applies the filtering rules, such as removing duplicate entries, excluding extremely low-quality scores, and ensuring diversity across trusted domains.

Once the relevant documents are selected, the system does not immediately pass them to reasoning model. Instead, each document is processed carefully. The content is segmented into manageable chunks so that specific passages can be analyzed independently. In addition, metadata, such as source type, publication origin and trust level is attached to each chunk. This step is important because it prepares the documents for the next stage, where capability policies will control how the information is used. At this point, the retrieval stage is complete. The system has transformed a structured clinical query into a curated set of relevant medical evidence gathered from multiple sources. More importantly, the retrieved content is not treated as neutral text. It is organized, labeled, and prepared for controlled reasoning. This ensures that the next stages of pipeline can operate on well-structures and traceable information rather than raw and potentially unreliable data.

In this way, the retrieval stage act as a bridge between query formulation and safe reasoning. It ensures that the system gathers meaningful and relevant medical knowledge while maintaining organization and transparency, which are essential for building a secure and reliable medical AI system.

C. Capabilities

After retrieving information from external medical sources, the first step in ensuring safe and reliable reasoning is to assign capabilities to every piece of information. Capabilities are essentially metadata that describe the origin, trust level and allowed usage of data. They act as "tags" that travel with the information throughout the system, guiding how it can be stored, merged, or used in reasoning.

For example, a peer reviewed article from PubMed may be labeled as high-trust scientific evidence, while posts from patient forums or social health networks might be tagged as low-trust anecdotal information. If a document contains sensitive patient information, it receives additional restrictions specifying who is allowed to access or act upon it. Capabilities are also granular. Each value or variable derived from a document – such as symptom, lab results, or treatment recommendation – carries its own tag. These tags track the source of the data, any intermediate transformations, and the allowed readers or tools that can access it. For instance, a lab result extracted from a patient forum post might be restricted from being directly used in medical recommendation, whereas the same type of information from a clinical guideline could be fully usable.

By attaching capabilities at the earliest stage, the system preserve a clear record of data provenance and reliability. In short capabilities act as a foundation for our system's safe reasoning. they make it possible to merge and process heterogeneous medical sources responsibly, track the origin and quality of information, and enforce fine-grained rules on what can and cannot be used in generating responses.

D. Knowledge Consolidation and Query-Specific Databases

Once capabilities have been assigned to all retrieved documents – capturing their source, trust level, and any restrictions, the next step is to consolidate and organize the information in meaningful way. Capabilities serve as a guide during this process, ensuring that only appropriate and reliable data is merged while preserving provenance and restrictions. Instead of treating each retrieval cycle as temporary, the system merges the selected documents and builds a structured knowledge base around the specific type of query. In other words, for every clinical query, the system gradually creates its own focused database. To explain this more clearly, let's look at the following cases:

For the Acne label, user discuss persistent breakouts, rosacea-like redness, concerns about isotretinoin side effects, and treatment failures. Even though the wording is different, the core theme is similar: acne management and treatment concerns. Guided by capabilities system

retrieves relevant medical guidelines and research, then merges them into an acne-specific database. Over time, this database grows to include evidence on isotretinoin, topical treatments, rosacea overlap, and mental health considerations.

For the appendicitis label, users describe ruptured appendix complications, cheonic diarrhea after surgery, wound infections, and fear of delayed rupture. these posts share a common theme of appendicitis and post-surgical outcomes. The system uses capabilities to ensure that reliable, medical sound information is selected, consolidating about appendectomy recovery, complications, bowel changes, and warning signs into an appendicitis-specific database, which is updated as similar queries appear.

Similarly, for the cancer label, users worry about small, dark skin spots and whether they might be dangerous. Using capabilities to prioritize trusted sources like dermatology guidelines, the system gather information about mole assessment and melanoma signs. The information is then organized into a skin-cancer-related database that supports future queries about suspicious lesions.

After retrieval, all relevant documents are connected to that theme are then combined and stored together. Over time, this creates a query-specific knowledge repository. As new queries are processed in the future, the system does not start from the scratch, Instead it updates the corresponding database by adding newly retrieved relevant documents removing outdated or low quality entries, and refining the stored content. This means the knowledge base becomes richer and more precise with use. The more frequently a particular query type appears, the more complete and robust its associated database becomes.

This structure creates smaller, focused databases instead of one large and messy collection of information. Each database is linked to a specific query or medical category. For example, there can be one database for acne-related questions, another for appendicitis cases, and another for cancer concerns and so on. This design has clear benefits. First, it makes the system faster and more efficient, because when a new query comes in, it can first look at the already built database for that category instead of searching everything from the scratch.. Second it keeps answers more consistent, since similar cases rely on the same growing pool of information. Third it improves transparency, because the system can track which database contributed to a particular response.

E. Security Policies

While capabilities describe what the data is, security policies define what can be done with it. In our system, policies enforce rules such as:

- High-trust evidence can be used for reasoning and answering queries, while low-trust anecdotal content may only provide context.
- Sensitive or personally identifiable information cannot be used without explicit user permission.
- Conflicting information is resolved by prioritizing high-trust sources, with uncertainties clearly flagged.
- Database updates must preserve capabilities, ensuring that provenance and restrictions are maintained even after consolidation.

These policies are expressed programmatically, allowing the system to enforce fine-grained rules. They prevent low-trust or restricted data from influencing outputs in unsafe ways, while allowing high-quality information to guide reasoning.

F. Interpreter

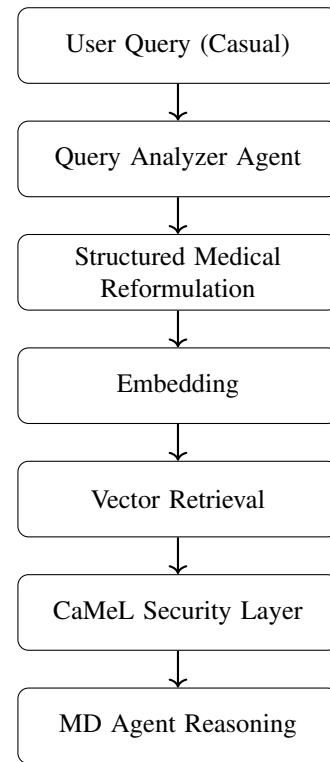


Fig. 2: CaMeL Methodology Architecture

VI. DATASET

For the Queries dataset, we are using the [5] dataset that contains patient descriptions and the classified disease of the description. This dataset contains **11104** patient descriptions.

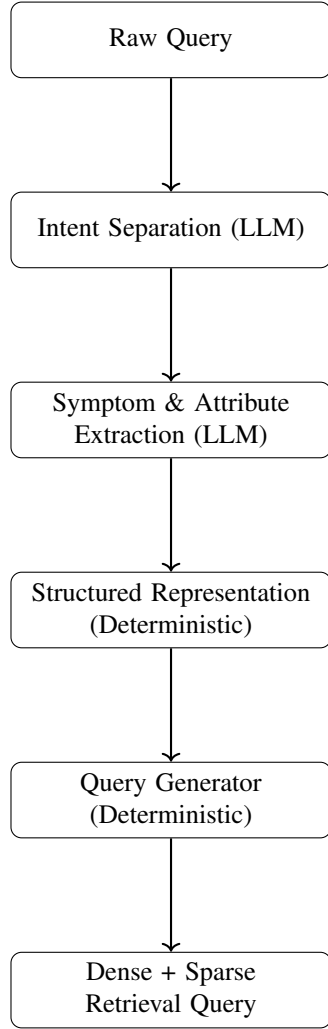


Fig. 3: Query Formulation Pipeline

VII. DATA CURATION

As we only have queries dataset available but not relevant data against the queries, that is the requirement of RAG. So, we are creating the dataset by scraping the following sources:

- 1) PubMed
- 2) Patient Info
- 3) HealthBoards
- 4) HealthShared
- 5) SocialHealthNetwork

VIII. SCRAPING STRATEGY

To scrape data accurately, we follow the following steps:

A. Data Analysis

Understand the dataset:

- 1) Count Descriptions per label

- 2) Group the diseases into categories

To scrape the relevant data per description, we need to extract the information important from a medical point of view. This includes:

- 1) Symptoms
- 2) Anatomical Location
- 3) Onset
- 4) Frequency
- 5) Triggers
- 6) Age
- 7) Gender
- 8) Pregnancy Status
- 9) Comorbidities
- 10) Clinical Intent
- 11) Clinical Flags
- 12) Duration
- 13) Severity

IX. PERFORMANCE ANALYSIS

A. Experimental Setup

- 1) OS Ubuntu 24.04.3 LTS
- 2) Processor AMD Ryzen™ 7 6800H
- 3) RAM 16 GB
- 4) LLM GPT 4o mini

B. Experimental Results

X. DISCUSSION:

11th December'2025

Based on the experimental results, we can dive into the implementation of guardrails and policies for detecting injection and performing countermeasures in the following directions:

- 1) User, can be malicious [low priority]
- 2) Dataset / Data Center, retrieved data from RAG [high priority]
- 3) Both user and dataset [low priority]

Q: How to implement this on RAG?

A: Rough idea

- 1) detecting malicious instruction from retrieve chunks

XI. CONCLUSION AND FUTURE DIRECTION

REFERENCES

- [1] P. Zhou, Y. Feng, and Z. Yang, "Privacy-aware rag: Secure and isolated knowledge retrieval," *arXiv preprint arXiv:2503.15548*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.15548>
- [2] Z. Jeddi and A. Bohr, "Remote patient monitoring using artificial intelligence," in *Artificial intelligence in healthcare*. Elsevier, 2020, pp. 203–234.
- [3] Y. Shen, J. Shao, X. Zhang, Z. Lin, H. Pan, D. Li, J. Zhang, and K. B. Letaief, "Large language models empowered autonomous edge ai for connected intelligence," *IEEE Communications Magazine*, vol. 62, no. 10, pp. 140–146, 2024.
- [4] S. Willison, "The dual llm pattern for building ai assistants that can resist prompt injection," *Online: https://simonwillison.net/2023/Apr/25/dual-llm-pattern*, 2023.

- [5] S. Shaikh, M. Y. Khan, and M. S. Nizami, "Using patient descriptions of 20 most common diseases in text classification for evidence-based medicine," in *2021 1st Mohammad Ali Jinnah University International Conference on Computing (MAJICC)*, 2021, pp. 1–8.

XII. PRIVACY-AWARE RAG: SECURE AND ISOLATED KNOWLEDGE RETRIEVAL - SUMMARY CHATGPT